

1) What is Spring Boot?

Spring Boot is an **open-source framework** built on top of the Spring Framework that simplifies the development of Java-based applications. It is designed to make it easier to create stand-alone, production-grade Spring-based applications with minimal configuration.

Spring Boot was developed by Pivotal Software(now the part of VMware), It was first released in 2014.

2) Where is the use of Spring Boot?

- i)Web Application
- ii)RESTful Web Services
- iii)Microservices iv)Batch Processing
- v)Integration with Other Technologies
- vi)Testing
- vii)Monitoring and Management

3) Spring Boot Core Principle.

- ii) **Convension Over Configuration:-** If you follow the standard rules or naming conventions, Spring Boot will automatically configure things for you — no need to write extra setup code.
- iii) **Auto-Configuration:-** Spring Boot automatically sets up your application based on the libraries you have added and the settings it finds — you don't have to configure everything manually.
- iv) **Embedded Servers:-** Spring Boot comes with a built-in (embedded) web server, so you don't need to install or configure a separate server like Tomcat or Jetty — it runs right inside your application.

2) What are the advantages of Spring Boot?

The main advantages of Spring Boot include:

Auto-configuration: Automatically configures Spring and third-party libraries based on project dependencies.

Standalone: Spring Boot applications can run as standalone Java applications.

Production-ready: Includes features like metrics, health checks, and externalized configuration.

Microservice-ready: Ideal for building microservices due to its lightweight and modular design.

3) How does Spring Boot simplify development?

Spring Boot simplifies development by:

- **Reducing Configuration:** Provides default configurations and auto-configuration options.
- **Embedded Servers:** Includes embedded servers like Tomcat, Jetty, or Undertow, eliminating the need for external server setup.
- **Spring Boot Starter Projects:** Offers pre-configured dependency sets for common tasks.

4) What are Spring Boot Starters?

Spring Boot Starters are a set of convenient dependency descriptors you can include in your application. They simplify the process of adding commonly used dependencies.

or

Spring Boot starters are pre-configured dependency sets for common application needs. They simplify dependency management by grouping related dependencies into a single artifact.

Example of including a Spring Boot Starter in pom.xml:

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

6) What is the use of Spring Boot Starters?

i) Simplifies Configuration:- Starters automatically include the right libraries and settings we need, so that we write code faster.

ii) Boost Productivity:- Saves time and effort by providing sensible defaults and configuration, allowing developers to focus on writing business logic rather than managing dependencies.

iii) Simplifies Configuration:- Ensures that all included dependencies are compatible with each other and the Spring Boot version being used, reducing the risk of conflicts.

7) Which are the most important Spring Boot Starters?

i) Spring-boot-starter:- It is the core starter that includes auto configuration support, logging, and YAML processing.

ii) Spring-boot-starter-web:- It provides dependencies to build web applications, including Spring MVC, Restful Services, and embedded Tomcat.

iii) Spring-boot-starter-data-jpa:- It provide dependencies for Spring Data JPA, Hibernate and database connection pool.

iv) Spring-boot-starter-security:- It includes Spring Security and it's dependencies for securing application.

v) Spring-boot-starter-thymeleaf:- It integrates the Thymeleaf templating engine with Spring MVC.

vi) Spring-boot-starter-test:-It includes testing libraries like Junit, Hamcrest and Mockito.

8) What is the difference between dependent jar file and relevant jar file?

Dependent JAR:- A JAR file that your application must have to run or compile — like Spring libraries if you're using Spring Boot. -Relevant

Relevant JAR:- A JAR file that is related or useful to your project, but not always required — like an email library if your app sends emails.

9) Define @SpringBootApplication?

i)@Configuration:- This annotation in Spring indicates that the class can be used to define bean configuration within the Spring application context.

ii)@ComponentScan:- It tells Spring to scan the specified package and it's sub packages for component, configuration and service to manage.

iii)@EnableAutoConfiguration:- It automatically configures the Spring application based on its dependencies and the contents of the classpath. -@SpringBootApplication annotation serves as the "Entry point for a SpringBoot application.

10) Different Categories of Starters.

i)Cloud Starter

ii)Web Starter

iii)Data Store Starter

iv)Spring Core Starter

v)Netflix Oss Starter

vi)Message Starter

vii)Batch Starter

10) What is “static folder in Spring Boot Project?

The “static folder” is typically used to store static web resources such as HTML, CSS, JavaScript, Images and other assets. -These files are served directly by the Web Server without any processing by the backend application.

11) What is “templates folder” in Spring Boot Project?

The “templates folder” is commonly used to store dynamic web pages or templates, Which are typically processed by a templating engine like Thymeleaf, FreeMarker or Velocity. -These templates are rendered dynamically by the server side code before being sent to the client.

5) What is the Spring Initializr?

The Spring Initializr is an online tool that helps generate a Spring Boot project with the desired configuration and dependencies. You can access it at start.spring.io.

6) What is Spring Boot dependency management?

Spring Boot manages dependencies by providing default versions and configurations for commonly used libraries through its starter dependencies. It helps avoid version conflicts and simplifies dependency management.

7) What are the Spring Boot properties?

Spring Boot properties are configuration settings that can be defined in application.properties or application.yml files to customize the behavior of your application.

8) What is Spring Boot Actuator?

Spring Boot Actuator provides production-ready features such as monitoring, metrics, and health checks. It helps in managing and monitoring your application in production environments.

9) How to connect Spring Boot to the database using JPA?

To connect Spring Boot to a database using JPA:

- i. Add JPA and database dependencies to pom.xml spring-boot-starter-data-jpa and any other dependency according to DB.
- ii. Configure database settings in application.properties:

spring.datasource.url=jdbc:h2:mem:testdb

spring.datasource.driver-class-name=org.h2.Driver

spring.datasource.username=sa

spring.datasource.password=password

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
- iii. Create JPA entity and repository.

10) What is @Controller and @RestController and @RequestBody? Or

What's a difference in it @Controller and @RestController and @RequestBody

@Controller is used in Spring MVC to handle web requests and return a *view name* (like a JSP, Thymeleaf, or HTML page). It is mainly used for building traditional web applications where the response is a UI page. By default, methods in @Controller do not return JSON or XML unless they are combined with @ResponseBody. Its main job is to accept requests, prepare model data, and forward it to a view for rendering.

@RestController is a specialized version of @Controller used for RESTful web services. It combines @Controller and @ResponseBody, which means every method automatically returns data in JSON or XML format instead of a view. It is commonly used

in APIs where the client is a mobile app, frontend framework (like React or Angular), or another service. In short, `@Controller` is for web pages, while `@RestController` is for APIs.

@RequestBody is used to read the HTTP request body and convert it into a Java object automatically using Jackson or another converter. It is mostly used with POST or PUT requests to get JSON input from the client. For example, when a client sends a JSON object, `@RequestBody` maps that JSON directly to a Java class. This makes it easy to handle API input in a clean and type-safe way.

11) What is Conditional annotation?

Conditional annotation in Spring Boot's auto-configuration determine whether specific configuration should be applied based on conditions like the presence of classes, properties or beans. This enables automatic configuration tailored to the application's environment and dependencies.

12) What is Stereo type annotation?

Multiple annotations doing the same work of configuring java class as spring bean are called stereo type annotation.

13) Define @Named, @Inject and @Resource?

i)@Named:- To configure the java class as spring bean and also to resolve ambiguity problem.

ii)@Inject:- Alternate to `@Autowired`

iii)@Resource:- Alternate to `@Autowired+@Qualifier`

14) What happens if we configure service, DAO classes using @Component annotation?

Yes, we can configure....But additional behaviour required for the spring bean should be added manually/explicitly.

15) What is RestTemplate?

RestTemplate is a helper class provided by Spring that makes it easy to call REST APIs (HTTP services) from your Spring Boot application.

16) What is WebClient?

WebClient is a modern and powerful tool in Spring used to make HTTP calls to other APIs, just like RestTemplate, but with more features and better performance.

17) Define @ConfigurationProperties?

It's a way to read values from your application.properties or application.yml file and put them into a Java class automatically. - So you don't have to write @Value again and again for every property. You just create one class, and Spring fills in all the values for you.

18) If @Value and @ConfigurationProperties tries to inject two different values to same spring bean property then which will be taken as final value?

The @ConfigurationProperties annotation assigned/injected value will be taken as final value becoz @Value performs fields Injection right after spring bean instantiation where as @ConfigurationProperties performs Setter injection by calling the setter method...So setter method injected value overrides the field injection value.

19) How to configure user-defined properties file in spring boot application?

Using @PropertySource annotation as spring bean class level annotation.

20) if we place both application.properties and application.yml having same keys and different values can you tell me what happens?

The value given to application.properties will override the values given in application.yml

21) What is Profiles?

A profile is like a label or a tag that tells Spring which settings or code to use depending on the environment.

- i) Development (dev)
- ii) Testing (test)
- iii) Production (prod)

22) If one target class is having multiple possible dependents to inject then how can we inject specific dependent of our choice without disturbing the source code of the Application/Project?

Using spring boot profiles(Best and recommended)

23) If we activate two different profiles using application.properties and using system property then which will be taken as the final profile?

The "profile specified" in the System property will be taken as the final value.

24) How many ways are there to activate profiles in spring boot app?

There are multiple ways to use...but mainly 3 approaches can be considered.

- i)using application.properties/yml(Best)
- ii)using System property(-Dspring.profiles.active=dev)
- iii)using Programatic approach(by using the methods of SpringApplication class.

25) What is the difference between fallback profile and default profile and also child profile?

Fallback Profile:-The profile that executes when requested active profile is not available.

default profile:-The profile that executes when there is no activate profile in the application.

child profile:-The profile that is included in application.properties by specifying the profile name to reuse keys and values of specific profile in application.properties indirectly as fallback profile.

26) **What is Runners?**

Runners are special blocks of code that run automatically when your Spring Boot application starts — right after everything is set up.

27) **What is the difference between placing static block in spring bean class and working with Runner class?**

Static block of Spring Bean class

It is a Java feature and can be used in both Spring and Spring Boot applications.

Executes automatically when the IOC container loads the Spring Bean class.

Does not allow passing data or arguments from outside.

Non-static data cannot be accessed inside the static block.

Useful for class-level one-time execution logic.

Static block does not support exception propagation.

Runner Class

It is a Spring Boot feature and can be used only in Spring Boot applications.

Executes automatically during Spring Boot application startup via `SpringApplication.run()` method.

Allows passing data or arguments to the `run()` method.

Allows access to both static and non-static data.

Useful for application-level startup logic.

`run()` method of Runner class supports exception propagation.

What is the difference between **CommandLineRunner** and **ApplicationRunner**?

Both are one time execution classes/spring bean having run() method..But the way they get cmd line args to the run() is different.
-CommandLineRunner gets the given cmd line args in the form of String[] where as ApplicationRunner gets the given cmd line args in the form of ApplicationArguments object where we can categorize the cmd line args into optional and non-optional args.

28) what is Lombok & @NoArgsConstructor, @AllArgsConstructor, @Data, @Builder.?

Lombok is a Java library that reduces boilerplate code by generating common methods automatically at compile time. Instead of manually writing getters, setters, constructors, toString(), equals(), and hashCode(), Lombok lets you use simple annotations. This makes your code cleaner, shorter, and easier to maintain, while improving readability and development speed. Lombok works through annotations and an IDE plugin that shows the generated methods during development.

@NoArgsConstructor generates a constructor with no parameters, useful for frameworks like Hibernate and Spring that require a default constructor.

@AllArgsConstructor generates a constructor with parameters for all fields in the class.

@Data is a shortcut annotation that bundles multiple features: it generates getters, setters, toString(), equals(), hashCode(), and a required-args constructor.

@Builder implements the Builder Pattern, which helps in creating objects in a readable and flexible way, especially when there are many fields. For example, instead of passing many parameters in a constructor, you can build an object step by step using ClassName.builder().field1(x).field2(y).build().

Other useful Lombok annotations include:

@Getter / @Setter – Generates getter and setter methods for fields or classes.

@RequiredArgsConstructor – Creates a constructor for final and @NonNull fields only.

@ToString – Generates a toString() method.

@EqualsAndHashCode – Generates equals() and hashCode() methods.

29) We configure DAO class with @Service annotation and Service class with @Repository Annotation?

No, we should not configure a DAO class with @Service and a Service class with @Repository. That's a wrong usage of annotations. - @Repository → used for DAO (Data Access Object) classes. These classes interact with the database. - @Service → used for Service classes. These contain business logic.

30) If we place all the 4 datasources(Tomcat cp,hikari cp,apache dbcp2, apache dbcp) related dependencies at a time in spring boot App then what happens?

As per Algorithm priority the Hikari CP Data Source will be taken for AutoConfiguration.

31) What is the difference between getForEntity() and getForObject() methods?

getForEntity() return type is ResponseEntity which contains response body(output),headers,status code and etc. -

getForObject() return type is Object which gives only response body(output).i.e It does not give response headers,status code and etc.

32) What is Serializable interface?

In Java, Serializable is a marker interface used to tell the program. - "Hey, this object can be saved, sent, or converted into a format (bytes) that can be restored later."

32) Though, we are not adding any starters to Spring boot project, How does it gets multiple jar files as the maven/gradle dependencies to the Project?

In every Spring boot Project, One parent spring boot project will be imported from the maven central repo using tag. → Based on the parent spring boot project's version, lots of things will come to our spring boot project which is nothing but a child project. a) Basic spring, spring boot jar files and dependent, relevant jar files. b) required plugins c) required Project Properties.

33) Define Debugging the Application using Eclipse IDE?

Debugging is useful to feel application code execution line by line
-Using debugging, we can identify the errors, fix the errors
-Using Debugging, we can feel the data flow
-Using Debugging, we can understand the code written by others
-Using Debugging, we can enable monitoring/watching on the variable data, object data, and expression.

33) What is @RequestMapping and @PostMapping annotation in Spring Boot?

@RequestMapping is a general-purpose annotation used to map HTTP requests to controller methods. It can handle *any HTTP method* like GET, POST, PUT, DELETE, etc. You must specify the method explicitly if you want a specific type, for example:

```
@RequestMapping(value="/save", method=RequestMethod.POST)
```

It is flexible but more verbose.

@PostMapping is a shortcut annotation used only for HTTP POST requests. It is a specialized form of @RequestMapping that already has method = RequestMethod.POST internally. This makes the code cleaner and more readable, for example:

```
@PostMapping("/save")
```

It directly tells Spring that this API accepts POST requests.

34) How does Spring Boot simplify dependency management?

Spring Boot simplifies dependency management by:

Providing Starters: Pre-configured dependency sets for common use cases.

Auto-Configuration: Automatically configures application components based on the included dependencies.

Dependency Management: Uses dependency management to handle version conflicts and ensure compatibility.

35) What is the role of embedded servers in Spring Boot?

Embedded servers in Spring Boot allow applications to be run as standalone Java applications. Spring Boot includes embedded servers like Tomcat, Jetty, and Undertow, which simplifies deployment and reduces the need for external web server setup.

36) What are Profiles in Spring Boot?

Profiles in Spring Boot allow you to segregate application configuration and make it available only in specific environments. You can define different properties for different profiles (e.g., dev, test, prod)

Example of using profiles:

```
# application-dev.properties
```

```
server.port=8081
```

```
# application-prod.properties
```

```
server.port=80
```

37) Is it possible to change the port of the embedded Tomcat server in Spring Boot?

Yes, you can change the port of the embedded Tomcat server by setting the `server.port` property in `application.properties`.

38) Can we disable the default web server in the Spring Boot application?

Yes, you can disable the default web server in a Spring Boot application by using the `spring.main.web-application-type` property. Set it to none.

```
spring.main.web-application-type=none
```

39) How to disable a specific auto-configuration class?

You can exclude a specific auto-configuration class by using the `@SpringBootApplication` annotation with the `exclude` attribute:
`@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})`

40) Can we create a non-web application in Spring Boot?

Yes, you can create a non-web application in Spring Boot by setting the `spring.main.web-application-type` property to none in `application.properties`:

```
spring.main.web-application-type=none
```

41) What is the difference between RequestMapping and GetMapping?

`@RequestMapping`: Can be used to map HTTP requests to specific handler methods, supporting all HTTP methods (GET, POST, PUT, DELETE, etc.).

`@GetMapping`: A specialized version of `@RequestMapping` for handling GET requests.

42) How do you enable Actuator in the Spring Boot application?

To enable Actuator, add the `spring-boot-starter-actuator` dependency to your `pom.xml`. You can then access actuator endpoints such as `/actuator/health` and `/actuator/info`.

43) What is Swagger in Spring Boot?

Swagger is a tool used to document and test RESTful APIs. In Spring Boot, you can integrate Swagger using the springfox-swagger2 and springfox-swagger-ui dependencies.

44) How do you implement security in a Spring Boot application?

To implement security, add the spring-boot-starter-security dependency. Then, configure security settings using @Configuration and @EnableWebSecurity.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .anyRequest().authenticated()
                .and()
                .formLogin();
    }
}
```

45) What is Spring Data JPA, and how does it differ from Hibernate?

Spring Data JPA is a part of the Spring Data project that simplifies database access and integrates JPA with Spring. Hibernate is a JPA implementation. Spring Data JPA provides additional functionality like repositories and query methods to ease data access.

46) How do you use Spring Boot with Docker?

To use Spring Boot with Docker, create a Dockerfile:

```
FROM openjdk:17-jdk
COPY target/myapp.jar /myapp.jar
ENTRYPOINT ["java", "-jar", "/myapp.jar"]
```

Build and run the Docker image:

```
docker build -t myapp .
docker run -p 8080:8080 myapp
```

47) How do you configure multiple data sources in Spring Boot?

To configure multiple data sources, define multiple DataSource beans and use @Primary to indicate the default one:

```
@Configuration
public class DataSourceConfig {
    @Bean
    @Primary
    @ConfigurationProperties("spring.datasource.primary")
    public DataSource primaryDataSource() {
        return DataSourceBuilder.create().build();
    }

    @Bean
    @ConfigurationProperties("spring.datasource.secondary")
    public DataSource secondaryDataSource() {
        return DataSourceBuilder.create().build();
    }
}
```

48) How do you handle CORS in Spring Boot?

You can handle CORS by configuring it globally or at the controller level. For global configuration:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE");
    }
}
```

49) What are the differences between Spring Data JPA and Hibernate?

- **Spring Data JPA:** A part of Spring that simplifies data access and integrates JPA with Spring. It provides a repository abstraction and various query methods.
- **Hibernate:** A JPA implementation that provides ORM capabilities for mapping Java objects to database tables. Spring Data JPA can use Hibernate as the JPA provider.

50) How to implement caching in Spring Boot?

To implement caching, add the spring-boot-starter-cache dependency and enable caching with @EnableCaching:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

```
@Configuration
@EnableCaching
public class CacheConfig {
}
```

Use the `@Cacheable` annotation to cache method results:

```
@Service
public class MyService {
    @Cacheable("myCache")
    public String getCacheValue(String key) {
        // Method logic
    }
}
```

51) How to configure Spring Boot for asynchronous processing?

Enable asynchronous processing with `@EnableAsync` and use `@Async` for asynchronous methods:

```
@Configuration
@EnableAsync
public class AsyncConfig {
}
```

```
@Service
public class MyService {
    @Async
    public CompletableFuture<String> asyncMethod() {
        // Asynchronous logic
        return CompletableFuture.completedFuture("result");
    }
}
```

52) What is the purpose of `@ComponentScan` in the class files?

The `@ComponentScan` annotation is used to specify the packages to scan for Spring components, such as `@Component`, `@Service`, `@Repository`, and `@Controller`.

```
@Configuration
@ComponentScan(basePackages = "com.example.myapp")
public class AppConfig {
}
```

53) How do you enable HTTPS in a Spring Boot application?

To enable HTTPS, configure SSL in `application.properties` :

```
server.port=8443
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=password
server.ssl.key-store-type=PKCS12
```

Place the `keystore.p12` file in `src/main/resources` .

54) What is Spring Boot WebFlux?

Spring Boot WebFlux is a reactive web framework that supports non-blocking and asynchronous processing. It is an alternative to the traditional Spring MVC framework for building reactive applications.

55) What is reactive programming in Spring Boot?

Reactive programming is a paradigm that deals with asynchronous data streams and the propagation of changes. Spring Boot supports reactive programming with the Reactor framework.

56) What is @EnableWebFlux?

@EnableWebFlux is a Spring annotation you put on a @Configuration class to activate Spring's reactive WebFlux stack. It imports the core WebFlux config so your app can use annotated controllers or functional endpoints for non-blocking, reactive web handling.

In short: add @EnableWebFlux when you're setting up a WebFlux app from scratch so Spring wires up the needed infrastructure for reactive web requests.

57) How do you integrate Spring Boot with RabbitMQ?

Add the spring-boot-starter-amqp dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

Configure RabbitMQ settings in `application.properties` :

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
```

Define a message listener:

```
@Component
public class MyMessageListener {
    @RabbitListener(queues = "myQueue")
    public void receiveMessage(String message) {
        System.out.println("Received: " + message);
    }
}
```

58) How do you use Spring Boot with Kubernetes?

Create a Docker image of your Spring Boot application and deploy it on Kubernetes using a YAML configuration file. Define a deployment and service in deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
```

```
template:
  metadata:
    labels:
      app: myapp
  spec:
    containers:
      - name: myapp
        image: myapp:latest
        ports:
          - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  ports:
    - port: 80
      targetPort: 8080
  selector:
    app: myapp
Apply the configuration with:
kubectl apply -f deployment.yaml
```

59) How do you secure a Spring Boot REST API using OAuth2?

Add the `spring-boot-starter-oauth2-client` dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Configure OAuth2 client settings in `application.properties`:

```
spring.security.oauth2.client.registration.google.client-id=your-client-id
spring.security.oauth2.client.registration.google.client-secret=your-client-secret
```

60) How to create custom annotations in Spring?

To create a custom annotation in Spring, we define an annotation using `@interface`, specify where it can be used with `@Target`, how long it is available with `@Retention`, and then process it using reflection, AOP, or an interceptor.

61) How do you configure global exception handling using `@ControllerAdvice`?

`@ControllerAdvice` or `@RestControllerAdvice`(for rest controller) is used to handle exceptions globally across all controllers. It centralizes exception handling logic in one class using `@ExceptionHandler`, improving consistency and readability."

For `@RestControllerAdvice` (Automatically adds `@ResponseBody`)

62) How does Spring Boot integrate with external configuration servers?

"Spring Boot integrates with external configuration servers using Spring Cloud Config, which allows applications to fetch configuration from a centralized server at startup or runtime."

How Integration Works (Short Steps)

1. Application connects to **Config Server** at startup.
2. Fetches configuration from **Git / Vault / File system / DB**.
3. Loads properties into Spring Environment.
4. Beans read values using @Value or @ConfigurationProperties.

63) What is spring-cloud-config-client and spring-cloud-config-server.

Spring Cloud Config Server (in short)

Spring Cloud Config Server is a **centralized configuration server** that stores all application configs (like application.properties, YAML files, DB configs, feature flags, secrets, etc.) in **one place**, usually a **Git repository**.

All microservices fetch their configuration from this single server.

Spring Cloud Config Client (in short)

A Config Client is any Spring Boot application that connects to the **Config Server** to fetch its config at startup (or refresh time).

It automatically loads properties from the server instead of its own local files.

Interview line:

"Config Client is a microservice that fetches its configuration from the Config Server instead of local application.properties."

How they work together (Quick Summary)

1. **Config Server** points to a **Git repo** containing config files.
2. Each microservice runs as a **Config Client**.
3. On startup, clients request:
`http://config-server/<app-name>/<profile>`
4. They receive their configuration dynamically.

5. Optional: Use @RefreshScope + /actuator/refresh to reload config without restart.

64) How do you implement scheduling in Spring Boot?

In Spring Boot, scheduling is implemented using @Scheduled and enabled using @EnableScheduling. It allows execution of tasks at a fixed interval, delay, or by cron expression.

65) How do you implement rate limiting and throttling in Spring Boot

Rate limiting and throttling in Spring Boot can be implemented using filters, interceptors, API Gateway, or libraries like Bucket4j or Resilience4j to restrict the number of requests from a client in a given time window.

66) what is @EnableMongoAuditing

@EnableMongoAuditing is used to turn on automatic auditing for MongoDB in a Spring application. Auditing means keeping track of when a record was created and when it was last updated. When this feature is enabled, Spring automatically fills these values whenever data is saved or modified in MongoDB. This removes the need for developers to manually handle timestamps and helps ensure that records always have correct creation and update information.

67) what is Rest Client in spring

A **REST Client in Spring** is a tool used to call external APIs or services from a Spring application, allowing your backend to communicate with other systems over HTTP. It sends requests like GET, POST, PUT, or DELETE to another server and receives responses such as data, status codes, or error messages, then converts those responses into usable Java objects. In simple terms, it works like a messenger that helps your application talk to other

applications or services without you having to handle complex networking logic.

68) **RestClient, WebClient ,RestTemplate ,HTTP Interface.**

RestTemplate is the older REST client provided by Spring for calling external APIs. It is synchronous and blocking, meaning the thread waits until the server responds. It is simple to use for basic API calls but is now considered outdated and is mainly kept for existing applications. It is not recommended for new projects because it lacks modern features and flexibility.

WebClient is the modern and powerful REST client provided by Spring. It supports non-blocking and asynchronous communication, which means the application does not have to wait for the response and can handle many requests efficiently. It is preferred for reactive and high-performance systems where scalability and speed are important. It also supports streaming and advanced request handling.

RestClient is a newer REST client introduced to replace RestTemplate with a cleaner and simpler design. It is still blocking like RestTemplate but provides a more modern and readable API. It is ideal when you want a simple synchronous REST client without moving into reactive programming.

HTTP Interface is the most high-level approach where you define REST calls as Java interfaces without writing any implementation. Spring automatically generates the client code for you behind the scenes. It makes code cleaner, easier to maintain, and looks similar to using Feign clients while internally using WebClient or RestClient.

69) What is webflux??

WebFlux is Spring's framework for building **reactive and non-blocking web applications**. It is designed to handle many requests at the same time using fewer system resources by not blocking threads while waiting for responses. Instead of stopping and waiting, the application continues working on other tasks and processes results when they arrive, making the system faster and more efficient under heavy load.

WebFlux is mainly used when you build high-performance APIs, real-time systems, or applications that need to handle thousands of users efficiently. It works well for streaming data, microservices, and applications that depend heavily on calling other services. Unlike traditional Spring MVC, which uses a blocking model, WebFlux uses an event-driven approach that improves scalability and responsiveness for modern cloud applications.

70) What is WebFilter (Spring WebFlux)?

A WebFilter is a Spring WebFlux interface used to intercept HTTP requests and responses in the reactive pipeline *before* they reach a handler and *after* the response is generated. It works similarly to Servlet Filters but is fully non-blocking and works with reactive types like Mono and Flux. You use it for tasks like logging, authentication, modifying headers, or request validation—all in a reactive, asynchronous way.

71) What is Mono<> (e.g., Mono<Boolean>)?

Mono is a reactive type from Project Reactor that represents a stream that emits 0 or 1 value asynchronously.

Mono<Boolean> means it will eventually produce one boolean value, or complete empty, or return an error. You get a Mono when performing operations like DB calls, API calls, authentication

checks, etc., where the result arrives asynchronously. It's the reactive alternative to returning a normal object but wrapped to support non-blocking execution.

72) What is SonarQube?

SonarQube is a self-hosted platform used for continuous inspection of code quality. It analyzes your code for bugs, vulnerabilities, code smells, duplications, and provides dashboards to track code quality trends. You install and run it on your own server.

73) What is SonarLint?

SonarLint is an IDE plugin (IntelliJ, VS Code, Eclipse) that gives real-time feedback as you write code. It highlights bugs, poor coding practices, and security issues inside the editor itself. Think of it as a "spell checker for code."

74) What is SonarCloud?

SonarCloud is the cloud-hosted version of SonarQube, offered as SaaS. You don't install anything; it directly integrates with GitHub, GitLab, Bitbucket, etc., to scan pull requests and show code quality results online. It's ideal for teams who want SonarQube features without hosting a server.

75) What is JsonNode class in java?

JsonNode in Java is a Jackson class used to read and work with JSON data in a tree structure. It lets you access JSON fields dynamically without creating Java classes, which is useful when the JSON format is unknown or changing.

76) What is RedisTemplate?

RedisTemplate is a Spring Data Redis class used to interact with a Redis server for performing operations like saving, retrieving, deleting, and updating data. It abstracts low-level Redis commands and provides high-level methods to work with Redis data types such as Strings, Hashes, Lists, Sets, and Sorted Sets. It handles serialization/deserialization and connection management internally, so developers can store Java objects directly in Redis. Essentially, **RedisTemplate is the main tool in Spring for performing Redis caching and data operations in a type-safe and convenient way.**

77) How do you handle exceptions in a Spring Boot application? (Short Para)

In Spring Boot, exceptions are handled by creating global error-handling mechanisms such as `@ControllerAdvice`, `@ExceptionHandler`, custom exception classes, and using Spring's built-in /error handling. You can return meaningful error responses using `ResponseEntity`, create standardized error formats, and log exceptions properly. This ensures readable messages for clients and clean centralized handling instead of try-catch everywhere.

78) How to handle exceptions using @ControllerAdvice and @ExceptionHandler? (Short Para)

`@ControllerAdvice` allows you to create a **global exception handler** that applies to all controllers in the application. Inside it, you define methods annotated with `@ExceptionHandler` to catch specific exceptions (like `NullPointerException`, `CustomException`, etc.) and return custom responses. This keeps exception logic separate from controller code and provides a clean, centralized way to format error messages, status codes, and logs.